FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Raven Gale Enriquez
Section:	AN31
Professor:	
Date:	Oct 2, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
```

```
}

# -------- Run Model ----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation ----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:

    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ⇩

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

if num_agents <= 0 or grid_size <= 0 or num_steps < 0:
    raise ValueError("Agents and grid size must be positive; steps cannot be negative.")

class RandomWalker(ap.Agent):
    def setup(self):
        self.position = (0, 0)
        self.path = []

    def step(self):
        # Choose one of four possible movement directions.
        direction = random.choice([
            (1, 0), (-1, 0), (0, 1), (0, -1)
        ])

        x, y = self.position

        # Keep agents inside the grid boundaries.
```

```
        x = max(0, min(self.model.p.grid_size[0] - 1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1] - 1, y + direction[1]))

        self.position = (x, y)
        self.path.append(self.position)

class RandomWalkModel(ap.Model):
    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions and record each starting point.
        for agent in self.agents:
            agent.position = (
                random.randint(0, self.p.grid_size[0] - 1),
                random.randint(0, self.p.grid_size[1] - 1)
            )
            agent.path = [agent.position]

        # Grid is non-toroidal, so agents cannot wrap around.
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

parameters = {
    "agents": num_agents,
    "grid_size": (grid_size, grid_size),
    "steps": num_steps
}

model = RandomWalkModel(parameters)
model.setup()

fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0] + 1))
ax.set_yticks(range(model.p.grid_size[1] + 1))
ax.grid(True)
ax.set_title("Random Walk Simulation")
ax.set_xlabel("X Position")
```

```
ax.set_ylabel("Y Position")

scat = ax.scatter([], [], s=200)

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    ax.set_title(f"Random Walk Simulation - Step {frame + 1}")
    return scat,

ani = animation.FuncAnimation(
    fig,
    update,
    frames=model.p.steps,
    blit=True,
    repeat=False
)

plt.show()
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
    - **The grid becomes crowded because many agents are sharing a small space. Their paths will overlap more often, and several agents may occupy the same or nearby positions.**
2. How does increasing the number of steps affect the **spread of agents**?
    - **Increasing the number of steps gives the agents more opportunities to move around the grid. As a result, they generally explore a larger portion of the grid and can spread farther from their starting positions. However, their movement remains random.**
3. Do the agents ever move outside the grid? Why or why not?

- **No. The agents are kept inside the grid by boundary checks in the program. The `max()` and `min()` functions restrict their x and y coordinates to valid grid positions, so an agent cannot move beyond an edge.**

4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
   - **The simulation can represent situations where individuals or particles move randomly. For example, it can model diffusion of particles, movement of people in a crowd, animal movement, or the spread of objects through an environment. By changing the movement rules, grid size, and number of agents, the simulation can represent different real-world conditions.**
5.

Answer the Questions ⇩